Lec-16

03/06/26 7:08 PM

```
String s = "   the sky   is    blue   ";
```

[i,j+1]

i to j+1

trim()

Coding

for (len=1; len<=s.length(); len++)

for (j=len; j<=s.length(); j++)

i=j-len

S.o (s.substring(i,j))

Kartik Bhaiya and Kanak Bhaiya are discussing a special type of number, which they call a **Coding Blocks Number (CB Number)**. A number qualifies as a CB Number based on the following criteria:

0 and 1 are not considered CB numbers.

The following prime numbers are always CB numbers:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29.

Any other number is a CB number if it is not divisible by any of the numbers listed in point (2).

0, 1 x CB

31, 37

1147

not Divisible

The Challenge

Kartik Bhaiya challenges Kanak Bhaiya with a problem:

He provides a **string of digits**, and Kanak Bhaiya must determine the maximum number of **CB numbers** that can be extracted from it while following these constraints:

1. Non-overlapping Substrings:

A CB number **cannot be a substring** or **superstring** of another chosen CB number.

Example: In **4991**, both **499** and **991** are CB numbers, but we can choose only **one** of them.

2. Valid Substring Selection:

The CB number must be a **contiguous substring** of the given string.

Example: In **481**, we **cannot** select **41** as a CB number because **41 is not a contiguous substring** of "481."

3. Maximization Goal:

Since multiple solutions may exist, the goal is to find the **maximum number** of CB numbers that can be extracted from the given string.

Kanak Bhaiya has a class of Launchpad students to teach and needs help solving this challenge. Assist him in finding the solution.

81615 → CB number

4991, 991, 499

481, 48, 81

number

81615

8161, 816, 81, 8, 161, 16, 1, 1615, 61, 615, 6

1 2 3 4 5

12, 23, 34, 45

123, 234, 345

1234, 2345

12345

8 1 6 1 5

0 1 2 3 4

```java
public static int count_cb_number(String s) {
        int c = 0;
        boolean[] visited = new boolean[s.length()];
        for (int len = 1; len <= s.length(); len++) {
            for (int j = len; j <= s.length(); j++) {
                int i = j - len;
                long num = Long.parseLong(s.substring(i, j));// i to j-1
                if (isCbnumber(num) && isnotvisited(visited, i, j - 1)) {
                    // marked
                    for (int k = i; k < j; k++) {
                        visited[k] = true;
                    }
                    c++;
                }

            }
        }
        return c;
    }
```

```java
public static boolean isnotvisited(boolean[] visited, int si, int ei) {
        // TODO Auto-generated method stub
        for (int i = si; i <= ei; i++) {
            if (visited[i]) {
                return false;
            }
        }
        return true;
    }
```

len=3

127

len=1

j=1 → i=0

1020hellotrueoky

Thread

Process